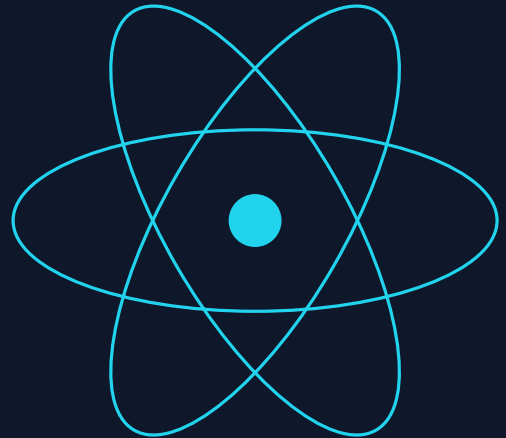




10-DAY INSTRUCTOR-LED TRAINING PROGRAMME

React Development

Beginner to Professional

Course Outline and Table of Contents

Level: Beginner, no prior programming experience required

Duration: 10 days, instructor-led, theory plus hands-on labs

Stack: JavaScript, TypeScript, React 19, React Router, TanStack Query, Tailwind CSS, Next.js, Supabase, Vitest, Playwright

Capstone: TaskBoard, a full-stack team task manager built across all 10 days

Course Overview

This programme takes delegates with no programming background from the fundamentals of the web and JavaScript through to building, testing and deploying a production-grade, full-stack React application. Every day combines short theory sessions with guided labs, and each lab adds a feature to a single capstone application, so delegates finish the course with a complete, deployed project in their own GitHub account.

Duration	10 days (approximately 60 contact hours)
Delivery	Instructor-led, classroom or virtual
Level	Beginner to advanced
Target audience	New developers, career changers, graduates, and IT staff moving into front-end or full-stack development
Prerequisites	Basic computer literacy: managing files and folders, installing software, using a web browser. No programming experience required.
Format	Approximately 40% theory and demonstration, 60% hands-on labs
Technology baseline	React 19, TypeScript, Vite, React Router, TanStack Query, Zustand, Tailwind CSS, Next.js (App Router), Supabase, Vitest, React Testing Library, Playwright, GitHub Actions, Vercel

Learning Outcomes

On completion of this programme, delegates will be able to:

- Set up a professional development environment with Node.js, VS Code, Git and GitHub
- Write modern JavaScript and TypeScript, including asynchronous code and API calls
- Build React user interfaces from reusable, well-structured components
- Manage local, shared and server state using hooks, Context, Zustand and TanStack Query
- Create multi-page applications with client-side routing and protected routes
- Build accessible, responsive, validated forms and interfaces with Tailwind CSS and component libraries
- Apply React 19 features including Actions, useActionState, useOptimistic and the use() API
- Build full-stack applications with Next.js Server Components, Server Actions and a Supabase backend
- Optimise rendering performance, write automated tests, and deploy through a CI/CD pipeline

Capstone Project: TaskBoard

TaskBoard is a team task manager that grows with the course. Delegates start with a static list of tasks on Day 3, add interactivity, persistence, TypeScript and routing through Days 4 to 8, migrate it to a full-stack Next.js application with authentication and a Postgres database on Day 9, and test and deploy it to production on Day 10.

Programme at a Glance

Day	Theme	Key outcome
1	Web and JavaScript Essentials	Working dev environment, first JavaScript programs
2	Modern JavaScript and First React App	ES6+, async code, Vite project scaffolded
3	React Fundamentals	JSX, components, props, lists
4	State and Interactivity	useState, events, controlled forms
5	Effects, Refs and Custom Hooks	Data fetching and reusable hook logic
6	TypeScript and Routing	Typed components, multi-page app
7	State Management and Server State	Context, useReducer, Zustand, TanStack Query
8	Styling, Forms and React 19 Features	Tailwind, validated forms, Actions, accessibility
9	Full-Stack React with Next.js	Server Components, Server Actions, Supabase auth and data
10	Performance, Testing and Deployment	Optimised, tested app live in production

Indicative Daily Schedule

08:30 to 09:00	Recap of previous day and daily knowledge check
09:00 to 10:30	Theory and live demonstration
10:30 to 10:45	Break
10:45 to 12:30	Guided lab
12:30 to 13:15	Lunch
13:15 to 14:45	Theory, demonstration and guided lab
14:45 to 15:00	Break
15:00 to 16:15	Capstone lab: TaskBoard feature build
16:15 to 16:30	Review, questions and take-home practice

Timings are indicative and can be adjusted to suit the venue and cohort.

DAY 1

Web and JavaScript Essentials

Objective: Set up a professional development environment and write first JavaScript programs.

Module 1.1 How the Web Works

- The roles of HTML, CSS and JavaScript in the browser
- Client and server, HTTP requests and responses
- What React is, and the problems it solves

Module 1.2 Developer Environment

- Installing Node.js (LTS) and npm
- VS Code setup and recommended extensions
- Terminal basics on Windows and macOS
- Git and GitHub fundamentals: init, add, commit, push

Module 1.3 HTML and CSS Refresher

- Semantic HTML elements
- CSS selectors and the box model
- Flexbox and Grid layout basics
- Using browser DevTools to inspect and debug

Module 1.4 JavaScript Fundamentals

- Variables with let and const, data types
- Operators, conditionals and loops
- Functions and scope
- Arrays and objects
- Debugging with the console and breakpoints

Hands-on Labs

- Lab 1.1: Environment setup and first GitHub repository
- Lab 1.2: Build a static profile page with HTML and CSS
- Lab 1.3: JavaScript exercises: tip calculator and grade checker

DAY 2

Modern JavaScript and Your First React App

Objective: Write the modern JavaScript that React relies on and create a first React project.

Module 2.1 Modern JavaScript (ES6+)

- Arrow functions and template literals
- Destructuring objects and arrays
- Spread and rest operators
- Optional chaining and nullish coalescing

Module 2.2 Working with Data

- Array methods: map, filter, find, reduce
- Immutability and why it matters in React
- JSON and data shapes
- ES modules: import and export

Module 2.3 Asynchronous JavaScript

- The event loop at a conceptual level
- Promises
- async and await
- The fetch API and error handling

Module 2.4 Hello React

- Component-based user interfaces
- Creating a React project with Vite
- Project structure walkthrough and the dev server
- How React renders and updates the page

Hands-on Labs

- Lab 2.1: Data transformation drills with array methods
- Lab 2.2: Fetch and display data from a public API in plain JavaScript
- Lab 2.3: Scaffold the TaskBoard capstone project with Vite and push to GitHub

DAY 3

React Fundamentals

Objective: Build user interfaces from reusable, composable components.

Module 3.1 JSX

- JSX syntax rules and embedding expressions
- Fragments
- Attributes, className and inline styles

Module 3.2 Components and Props

- Function components
- Passing and reading props
- The children prop and default values
- Component composition

Module 3.3 Rendering Data

- Rendering lists with map
- Keys and why they matter
- Conditional rendering patterns

Module 3.4 Thinking in React

- Breaking a design into a component tree
- Single responsibility components
- React Developer Tools
- File and folder conventions

Hands-on Labs

- Lab 3.1: Build a reusable Card and Button component set
- Lab 3.2: Render a product catalogue from static data
- Lab 3.3: TaskBoard: static task list UI from mock data

DAY 4

State and Interactivity

Objective: Make components respond to user input and manage changing data.

Module 4.1 useState

- State compared with props
- Updating state and re-rendering
- State as a snapshot
- Updater functions

Module 4.2 Handling Events

- Click, input, change and submit events
- Passing event handlers as props
- Preventing default browser behaviour

Module 4.3 Forms

- Controlled inputs
- Text inputs, selects, checkboxes and radio buttons
- Basic validation and error messages

Module 4.4 Managing State Well

- Immutable updates to arrays and objects
- Lifting state up
- Derived state and avoiding duplicated state

Hands-on Labs

- Lab 4.1: Counter, toggle and accordion exercises
- Lab 4.2: Shopping cart with add, remove and quantity controls
- Lab 4.3: TaskBoard: add, complete, edit and delete tasks

DAY 5

Effects, Refs and Custom Hooks

Objective: Synchronise components with external systems and package logic for reuse.

Module 5.1 useEffect

- What side effects are
- The dependency array
- Cleanup functions
- Common mistakes, and when an effect is not needed

Module 5.2 Fetching Data

- Loading, error and empty states
- Cancelling requests with AbortController
- Working with REST APIs

Module 5.3 useRef

- Accessing DOM elements: focus and scroll
- Storing values that do not trigger re-renders

Module 5.4 Custom Hooks

- Rules of hooks
- Extracting reusable logic
- Building useFetch and useLocalStorage

Hands-on Labs

- Lab 5.1: Search-as-you-type with debouncing
- Lab 5.2: Weather dashboard from a public API
- Lab 5.3: TaskBoard: persist tasks and load seed data from an API

DAY 6

TypeScript and Routing

Objective: Add type safety to React code and build multi-page applications.

Module 6.1 TypeScript Essentials

- Why TypeScript
- Basic types, interfaces and type aliases
- Union types and optional properties
- Introduction to generics

Module 6.2 TypeScript with React

- Typing props and children
- Typing state, events and refs
- Typing custom hooks
- Converting a JavaScript project to TypeScript

Module 6.3 React Router

- Client-side routing concepts
- Routes, links and nested layouts
- URL parameters and search parameters
- Not found pages

Module 6.4 Navigation Patterns

- Programmatic navigation
- Route loaders and actions overview
- Protected route pattern

Hands-on Labs

- Lab 6.1: Convert TaskBoard to TypeScript
- Lab 6.2: Multi-page TaskBoard: dashboard, project detail and settings
- Lab 6.3: Protected routes with a mock login

DAY 7

State Management and Server State

Objective: Choose the right tool for shared application state and remote data.

Module 7.1 Context API

- The prop drilling problem
- Creating, providing and consuming context
- Theme and authentication context examples

Module 7.2 useReducer

- The reducer pattern
- Actions and dispatch
- Combining useReducer with Context

Module 7.3 Global State Libraries

- Designing a Zustand store
- Redux Toolkit overview and when to choose it

Module 7.4 Server State with TanStack Query

- Queries and query keys
- Mutations
- Caching, refetching and invalidation
- Optimistic updates

Hands-on Labs

- Lab 7.1: Theme switcher with Context
- Lab 7.2: TaskBoard state refactor with useReducer and Zustand
- Lab 7.3: TaskBoard data layer with TanStack Query against a mock REST API

DAY 8

Styling, Forms and React 19 Features

Objective: Build polished, accessible interfaces and use the latest React APIs.

Module 8.1 Styling Approaches

- CSS Modules
- Tailwind CSS utility-first styling
- Responsive design and dark mode

Module 8.2 Component Libraries

- shadcn/ui and Radix primitives
- Icon libraries
- Building a consistent design system

Module 8.3 Production Forms

- React Hook Form
- Schema validation with Zod
- Accessible error handling

Module 8.4 React 19 Features

- Actions and async transitions
- useActionState and useFormStatus
- useOptimistic
- The use() API and ref as a prop

Module 8.5 Accessibility

- Semantic markup and ARIA basics
- Keyboard navigation and focus management
- Colour contrast and screen reader testing

Hands-on Labs

- Lab 8.1: Restyle TaskBoard with Tailwind CSS and shadcn/ui
- Lab 8.2: Task creation form with React Hook Form and Zod
- Lab 8.3: Optimistic task updates with useOptimistic

DAY 9

Full-Stack React with Next.js

Objective: Build a full-stack application with server rendering, a database and authentication.

Module 9.1 Next.js Foundations

- Why use a React framework
- App Router file-based routing
- Layouts, pages, loading and error files

Module 9.2 Server and Client Components

- Server rendering compared with client rendering
- The "use client" boundary
- Fetching data in Server Components
- Caching and revalidation

Module 9.3 Server Actions and Mutations

- Server Functions
- Form handling with Server Actions
- Revalidating data after changes

Module 9.4 Suspense and Error Handling

- Streaming with Suspense
- Error boundaries
- Loading UI patterns

Module 9.5 Backend with Supabase

- Postgres tables and relationships
- Supabase client setup
- Email and password authentication
- Row Level Security basics
- Environment variables and secrets

Hands-on Labs

- Lab 9.1: Migrate TaskBoard to the Next.js App Router
- Lab 9.2: Connect TaskBoard to a Supabase database
- Lab 9.3: Add sign up, login and per-user task data

DAY 10

Performance, Testing and Deployment

Objective: Optimise, test and ship a production-ready application.

Module 10.1 Performance

- How React decides what to re-render
- memo, useMemo and useCallback
- React Compiler and automatic memoisation
- Code splitting with lazy
- Profiling with React Developer Tools

Module 10.2 Testing

- The testing pyramid
- Unit testing with Vitest
- Component testing with React Testing Library
- Mocking API calls
- End-to-end testing with Playwright

Module 10.3 Shipping to Production

- Linting and formatting with ESLint and Prettier
- Environment configuration
- Continuous integration with GitHub Actions
- Deploying to Vercel
- Monitoring and error tracking overview

Module 10.4 Professional Practice

- Structuring larger projects
- Code review practice
- Lighthouse performance and accessibility audits
- Next steps and continued learning paths

Hands-on Labs

- Lab 10.1: Unit and component tests for TaskBoard
- Lab 10.2: CI pipeline and production deployment
- Lab 10.3: Capstone demonstrations and peer code review

Delegate Requirements

Hardware

- Laptop running Windows 10 or 11, or macOS
- Minimum 8 GB RAM (16 GB recommended) and 20 GB free disk space
- Administrator rights to install software
- Reliable internet access for package installation and cloud services

Software (installed before Day 1 where possible)

- Node.js, current LTS release, with npm
- Visual Studio Code
- Git
- Google Chrome or Microsoft Edge with the React Developer Tools extension

Accounts (free tiers are sufficient)

- GitHub account (from Day 1)
- Supabase account (from Day 9)
- Vercel account (from Day 10)

Assessment Approach

- **Daily knowledge checks:** short quizzes at the start of each day covering the previous day's content
- **Lab reviews:** instructor review of lab completion and code quality throughout the week
- **Capstone demonstration:** each delegate presents their deployed TaskBoard application on Day 10 and takes part in a peer code review

Take-Home Practice

Because the programme covers a large amount of material for delegates new to programming, a short practice exercise is set at the end of each day. Completing these consolidates the day's learning and keeps delegates on pace for the capstone build.